

Arquitectura y modelo de datos

Proyecto integrador | Documentación técnica y académica

Meteorología y energía sintética en cinco ciudades de Colombia

Juan Sebastian Clavijo Correa

Institución Universitaria Digital de Antioquia
Infraestructura y Arquitectura para Big Data
30 de septiembre de 2026

Alcance verificado	Resultado
Periodo y grano	2025 completo; una ciudad por hora local
Integración	43.800 filas; 32 columnas limpias y 63 enriquecidas
Modelo final	SQLite: 6 tablas y una vista de reconstrucción
Continuidad	DuckDB heredado de EA1, Pandas y GitHub Actions

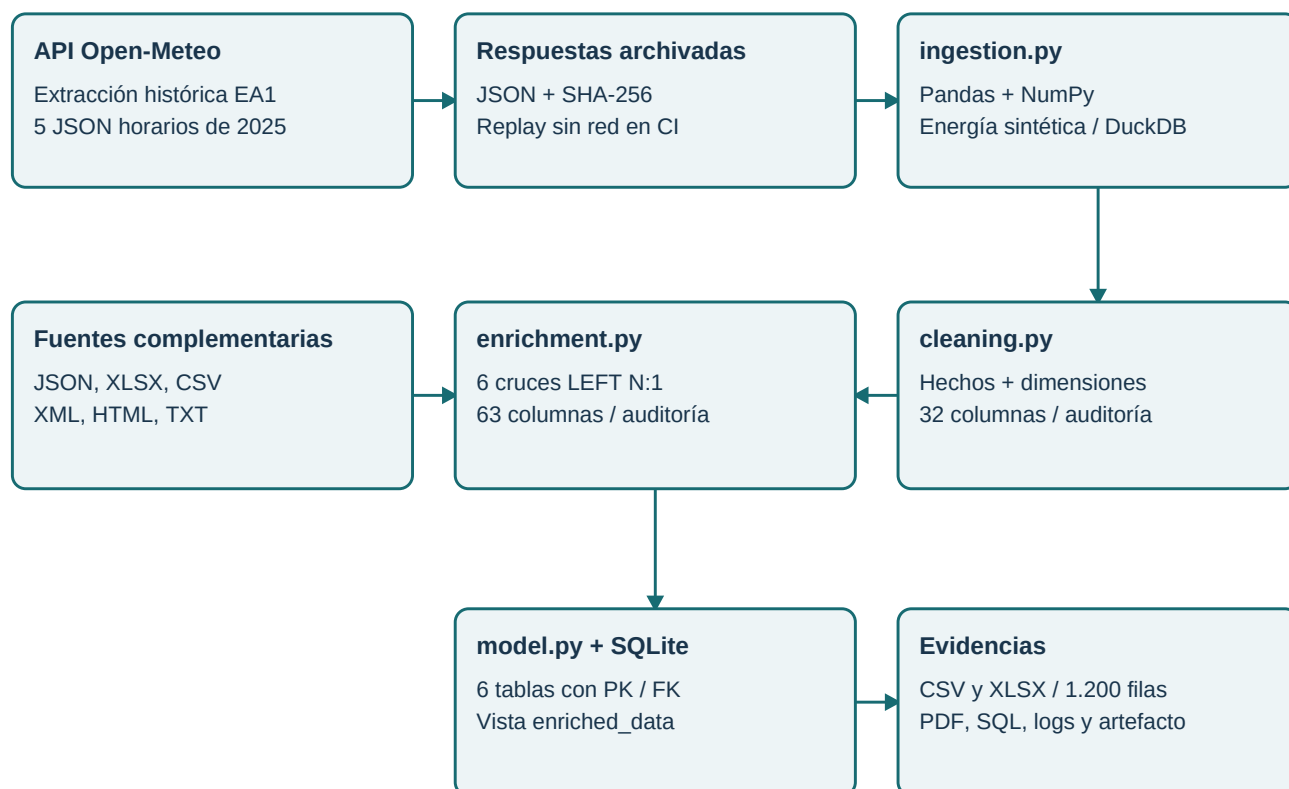
Este documento reúne las fases desarrolladas durante el curso y las contrasta con sus archivos de entrada, scripts y evidencias. La energía es una variable sintética con fines académicos; los resultados no describen consumo eléctrico medido ni prueban una relación causal con el clima.

Repositorio: [JSebastianCCorrea/documentacion_arquitectura_modelo_datos](#)

Lectura: arquitectura (2), ingesta (3), limpieza (4), enriquecimiento (5), modelo ER (6), diccionario físico (7-10), herramientas (11), automatización (12), resultados y recomendaciones (13), bibliografía (14).

1. Visión global de la arquitectura

El proyecto organiza datos meteorológicos horarios de Medellín, Bogotá, Cali, Barranquilla y Bucaramanga, y los combina con una simulación de energía. Su propósito es disponer de una base trazable para explorar patrones por ciudad y tiempo. El volumen permite ejecutar el proceso localmente y comprobar las reglas antes de trasladarlas a una infraestructura mayor.



GitHub Actions ejecuta la cadena en un runner y conserva los resultados durante 30 días.

Figura 1. Flujo completo. Elaboración propia a partir de los scripts del repositorio. El tramo API-JSON corresponde a la extracción de EA1; la ejecución automatizada actual comienza en las respuestas archivadas.

La entrada se conserva, la limpieza genera una versión tipada y el enriquecimiento añade contexto sin sustituir las variables originales. Después se materializa el modelo SQLite. Cada transición produce evidencias que permiten localizar una diferencia sin repetir solicitudes a proveedores externos.

La carpeta anterior se llamaba actividad_4, pero contenía la fase de enriquecimiento. En este informe se distinguen las fases por su función: EA1, ingesta; EA2, limpieza; tercera fase, enriquecimiento. La presente entrega integra y documenta esas implementaciones.

2. Ingesta y trazabilidad de origen

2.1 Extracción realizada en EA1

El script original consultó Historical Weather API de Open-Meteo mediante `requests.get`, con fechas 2025-01-01 a 2025-12-31, coordenadas de cada ciudad y `timezone=America/Bogota`. Solicitó `temperature_2m`, `relative_humidity_2m`, `precipitation` y `wind_speed_10m`. Cada respuesta quedó guardada como JSON. El API entrega productos meteorológicos modelados; la consulta de EA1 no fijó el parámetro `models`, por lo que no se atribuye toda la meteorología exclusivamente a ERA5 (Open-Meteo, s. f.).

Las coordenadas incorporadas a la base son las solicitadas para las ciudades. No deben confundirse con las coordenadas de la celda de la malla que devuelve el proveedor. Las respuestas originales conservan esa información y sus unidades.

2.2 Simulación de energía y carga analítica

EA1 asignó consumos base ilustrativos de 110, 130, 115, 125 y 100 a Medellín, Bogotá, Cali, Barranquilla y Bucaramanga. Multiplicó cada base por un factor horario (1,25 de 18 a 21 h; 1,15 de 7 a 9 h; 1 en otras horas), un factor térmico $1 + |T - 22| \times 0,015$, un factor de lluvia $1 + \text{precipitación} \times 0,005$ y ruido normal con media 1 y desviación 0,05. NumPy utilizó semilla 42 y se redondeó a dos decimales.

El resultado pasó a `staging_weather` y a las dimensiones `dim_city` y `dim_time`, relacionadas lógicamente con `fact_weather_energy`. La construcción original mediante `CREATE TABLE AS` no declaró claves primarias ni foráneas. Por eso la limpieza comprueba unicidad y ausencia de huérfanos antes de unir estas tablas.

Tabla de EA1 (DuckDB)	Filas	Función
<code>staging_weather</code>	43.800	Carga inicial: 11 columnas
<code>dim_city</code>	5	Identificador local y coordenadas
<code>dim_time</code>	8.760	Una hora por fila
<code>fact_weather_energy</code>	43.800	Ciudad/hora, meteorología y energía sintética

2.3 Reproducción en esta entrega

`src/ingestion.py` verifica SHA-256 de los cinco JSON y reconstruye la ingesta sin red. Mantiene el orden original de ciudades y horas para reproducir el ruido. `src/db/ingestion.db` es una copia inalterada de `analytics.duckdb` de EA1: su extensión `.db` no la convierte en SQLite. El replay se contrasta, valor por valor, contra la extracción de esa copia. El código histórico está en `docs/antecedentes/ingesta_ea1_original.py`.

3. Preprocesamiento y limpieza

src/cleaning.py conserva la lógica de EA2. Lee la tabla de hechos y sus dimensiones en modo de solo lectura, valida las claves y ordena por ciudad y tiempo. Asigna source_row para mantener la correspondencia con la salida original. Después normaliza los nombres de ciudad y convierte números, booleanos y fechas.

Control	Decisión aplicada
Clave ciudad-hora	Se eliminan duplicados exactos. Si una clave ciudad-hora tiene valores contradictorios, se rechazan todas sus versiones.
Valores ausentes	Se descartan claves inválidas. Las medidas imputables siguen la regla de mediana por ciudad, año y mes. Si falta lluvia o no hay donante para imputar, se rechaza la fila.
Rangos físicos	Latitud, longitud, humedad y medidas no negativas se validan antes de aceptar un registro.
Atípicos	Regla IQR por ciudad: marcas de revisión, sin recortar ni borrar automáticamente.
Zona horaria	Interpretación local America/Bogota y columna adicional observation_time_utc.
Transformaciones	Z-score de temperatura y energía por ciudad, humedad/100 y log1p de precipitación.

3.1 Lo observado en la entrada disponible

La base recibida ya tenía 43.800 filas sin nulos ni duplicados: en esta ejecución no se eliminaron filas, no se imputaron valores y no hubo rechazos. Se conservaron 6.240 filas con alguna marca de atípico. Aplicar controles no obliga a encontrar errores; el informe registra las operaciones realmente necesarias.

El CSV limpio contiene 32 columnas. Su huella coincide byte a byte con el archivo aprobado en EA2. Las muestras CSV y XLSX contienen 1.200 registros: hasta 20 por ciudad, año y mes, con semilla 42. La cobertura y los pesos de expansión permiten reconocer que se trata de una muestra estratificada de cuota fija.

3.2 Uso posterior de las variables

Los z-scores y las marcas IQR se calcularon con el año completo. Son útiles para describir esta base, pero no deben trasladarse sin revisión a un experimento predictivo. Primero habría que separar entrenamiento y evaluación por tiempo y ajustar las transformaciones solo con el entrenamiento. Esta entrega no entrena ni evalúa modelos de predicción.

Evidencias: src/static/auditoria/limpieza/cleaning_report.txt, cleaning_metrics.json, quality_profile.csv y rejected_records.csv. La copia histórica de auditoría de EA2 se conserva en ea2_original/ para distinguir el antecedente de la ejecución integrada.

4. Enriquecimiento de seis formatos

La entrada es el CSV limpio completo, no la muestra. Se carga en `cleaned_data` dentro de `cleaned.duckdb` y se lee con Pandas. Las seis fuentes están incluidas en el repositorio con sus manifiestos, instantáneas originales y hashes. Pandas permite lectores distintos y una operación merge con validación de cardinalidad (pandas development team, s. f.-a).

Archivo / formato	Clave de cruce	Filas fuente	Aporte
city_geography.json	Ciudad normalizada	5	País, GeoNames y elevación
calendar_2025.xlsx	Fecha local	365	Festivo y fin de semana
solar_daily_2025.csv	Ciudad + fecha	1.825	Radiación y duración solar
municipalities.html	Ciudad normalizada	5	Códigos y nombres DANE
thermal_parameters.xml	Ciudad normalizada	5	Referencia académica 18 °C
hour_bands.txt	Hora local	24	Franja del día

Cada unión es LEFT JOIN con `validate="many_to_one"`: todas las filas base se conservan y una clave repetida en una fuente detiene el proceso. La clave auxiliar de ciudad elimina tildes y diferencias de mayúsculas o espacios; el nombre original queda intacto. Las fechas se obtienen en Colombia antes de cruzar con fuentes diarias.

Los seis cruces encontraron correspondencia para las 43.800 filas; no aumentó el número de registros. Se añadieron 31 columnas: atributos, indicadores `matched_*`, conversiones de unidades y estado de integración. MJ/m²/día se divide por 3,6 para obtener kWh/m²/día; las duraciones en segundos se dividen por 3.600. Las diferencias con 18 °C son referencias ilustrativas, no índices de confort validados.

4.1 Procedencia y límites de interpretación

Hay cuatro conjuntos externos de tres proveedores: Open-Meteo/GeoNames, Nager.Date y DANE. XML y TXT son catálogos propios. El HTML se preparó a partir del servicio JSON oficial DANE, no de una página inventada. El calendario agrupa 18 festividades en 17 fechas. Bucaramanga conserva el año DIVIPOLA 2024 recibido por el servicio denominado 2025; no se corrigió ese dato sin evidencia.

Si falta correspondencia, `enrichment.py` conserva la fila con INCOMPLETE y no imputa los atributos externos. La materialización SQLite de esta entrega exige COMPLETE y detiene su publicación si encuentra faltantes. El dato solar del día completo es retrospectivo y no debe emplearse para anticipar una hora de ese mismo día.

5. Modelo lógico y relaciones

El modelo final es una constelación dimensional pequeña: dos tablas de hechos comparten dimensiones, y la dimensión temporal separa calendario y hora. La ciudad-hora identifica cada observación meteorológica; la ciudad-fecha identifica cada observación solar. Esta separación evita sumar 24 veces el valor diario que se repite en el archivo plano.

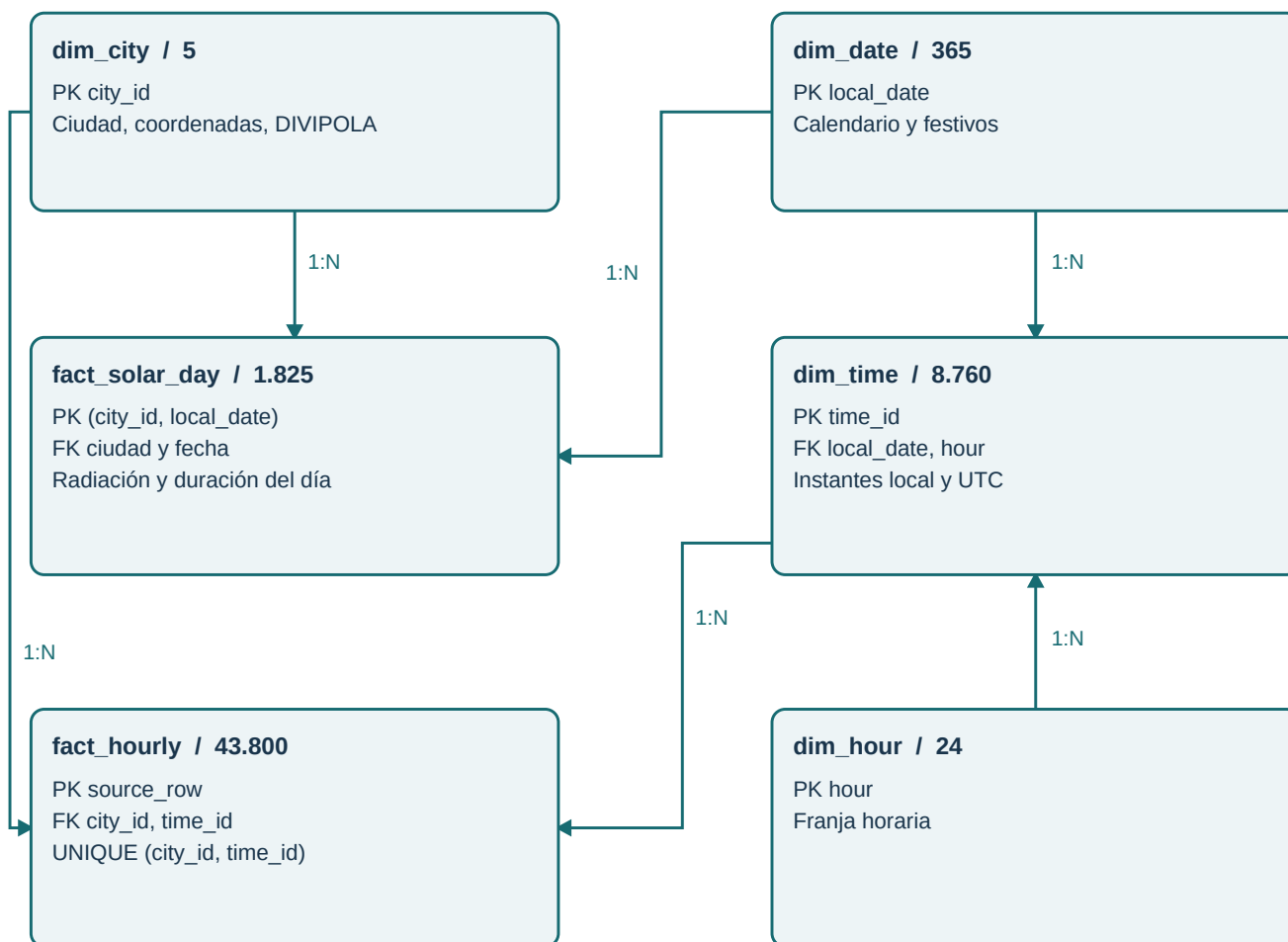


Figura 2. Modelo físico SQLite. Las flechas indican relación de padre a hijo 1:N. Cada hijo tiene una sola fila padre para la clave obligatoria. Elaboración propia; PK y FK corresponden al DDL ejecutado.

fact_hourly referencia dim_city y dim_time. dim_time referencia dim_date y dim_hour. fact_solar_day referencia dim_city y dim_date. No se enlazan directamente los dos hechos mediante una FK: la vista los integra por ciudad y la fecha obtenida desde dim_time.

model.py activa PRAGMA foreign_keys=ON, carga tablas STRICT en una base temporal y ejecuta foreign_key_check e integrity_check. Solo sustituye analytics.sqlite cuando la vista reconstruye las 63 columnas sin diferencias. Las claves, NOT NULL, UNIQUE y CHECK complementan los controles de Pandas (SQLite, s. f.-a).

6. Diccionario físico: ciudad

Los tipos siguientes proceden de PRAGMA table_info sobre la base creada. Todos los campos físicos son NOT NULL. Los booleanos se guardan como INTEGER con CHECK IN (0,1); los identificadores territoriales se conservan como TEXT para mantener ceros iniciales. Las unidades y definiciones completas de las 63 variables están también en DICCIONARIO_DATOS.md.

dim_city | 5 filas

Campo	Tipo / clave	Descripción
city_id	INTEGER / PK	Identificador dimensional local de EA1
city	TEXT	Barranquilla, Bogota, Bucaramanga, Cali o Medellin, con la escritura de EA2
latitude	REAL	Latitud solicitada en la extracción original
longitude	REAL	Longitud solicitada en la extracción original
country_code	TEXT	Código de país, CO
geoname_id	INTEGER	Identificador del punto en GeoNames
elevation_m	REAL	Elevación del punto geográfico retornado
matched_geography	INTEGER	La ciudad encontró correspondencia en el catálogo geográfico
municipality_code	TEXT	Código DIVIPOLA municipal
municipality_name	TEXT	Nombre municipal informado por DANE
department_code	TEXT	Código departamental
department_name	TEXT	Nombre departamental informado por DANE
divipola_year	INTEGER	Año del registro MPIO_NANO, conservado de la fuente
matched_municipalities	INTEGER	La ciudad encontró correspondencia DIVIPOLA
reference_temperature_c	REAL	Referencia ilustrativa de 18 °C
parameter_scope	TEXT	academic_illustration
matched_thermal	INTEGER	La ciudad encontró referencia térmica

city_id conserva el identificador local de EA1; no equivale a municipality_code. city y municipality_code tienen restricciones UNIQUE. Los códigos municipales y departamentales exigen cinco y dos dígitos, respectivamente. La dimensión representa únicamente las cinco ciudades incluidas, no un catálogo nacional completo.

7. Diccionario físico: calendario y tiempo

dim_date | 365 filas

Campo	Tipo / clave	Descripción
local_date	TEXT / PK	Fecha de la observación en Colombia
year	INTEGER	Año local: 2025
month	INTEGER	Mes local: 1 a 12
day	INTEGER	Día local del mes
holiday_name	TEXT	Nombre de festividad o No festivo
is_holiday	INTEGER	La fecha figura como festiva en el calendario
is_weekend	INTEGER	La fecha es sábado o domingo
matched_calendar	INTEGER	La fecha encontró correspondencia en el calendario
is_business_day	INTEGER	Fecha que no es festiva ni fin de semana según calendar

dim_hour | 24 filas

Campo	Tipo / clave	Descripción
hour	INTEGER / PK	Hora local: 0 a 23
day_period	TEXT	madrugada, mañana, tarde o noche
matched_hours	INTEGER	La hora encontró clasificación en el catálogo

dim_time | 8.760 filas

Campo	Tipo / clave	Descripción
time_id	INTEGER / PK	Identificador de la hora en la dimensión temporal de EA1
observation_time	TEXT	Instante de la observación, presentado en America/Bogota
observation_time_utc	TEXT	El mismo instante de observation_time, expresado en UTC
local_date	TEXT / FK	Fecha de la observación en Colombia
hour	INTEGER / FK	Hora local: 0 a 23
timezone	TEXT	America/Bogota

Las fechas se almacenan como texto ISO 8601. observation_time conserva el desplazamiento -05:00 y observation_time_utc el instante en UTC; timezone conserva America/Bogota. SQLite no tiene un tipo nativo TIMESTAMPTZ. El origen DuckDB sí lo utiliza en cleaned_data. (SQLite, s. f.-b).

8. Diccionario físico: hechos solares

fact_solar_day | 1.825 filas

Campo	Tipo / clave	Descripción
city_id	INTEGER / PK / FK	Identificador dimensional local de EA1
local_date	TEXT / PK / FK	Fecha de la observación en Colombia
shortwave_radiation_sum_mj_m2	REAL	Radiación de onda corta acumulada diaria
daylight_seconds	REAL	Duración diaria de luz
sunshine_seconds	REAL	Duración diaria de sol informada por la fuente
matched_solar	INTEGER	La combinación ciudad–fecha encontró correspondencia solar
radiation_kwh_m2_day	REAL	$\text{shortwave_radiation_sum_mj_m2} / 3.6$
daylight_hours	REAL	$\text{daylight_seconds} / 3600$
sunshine_hours	REAL	$\text{sunshine_seconds} / 3600$

8.1 Granularidad y agregación

La clave primaria compuesta (city_id, local_date) obliga a tener como máximo una observación solar por ciudad y día. Si las 24 filas horarias de un mismo día aportan valores solares distintos, model.py falla antes de escribir. La selección de filas únicas es válida porque primero se comprueba esa dependencia funcional.

La suma anual de radiación se consulta en fact_solar_day, agrupando por city_id. En enriched_data la medida diaria vuelve a repetirse para facilitar el análisis de cada hora: no se debe sumar directamente allí. La energía sintética horaria sí puede sumarse por periodos, siempre manteniendo su carácter simulado.

8.2 Integridad del dato diario

Los CHECK exigen radiación no negativa, duración de luz entre 0 y 86.400 segundos y sunshine_seconds entre cero y daylight_seconds. Las dos FK impiden cargar una ciudad o fecha inexistente. Las conversiones se conservan para lectura directa; sus fórmulas se prueban en la etapa de enriquecimiento.

Las fechas y variables meteorológicas describen 2025. La fecha de ejecución del pipeline, registrada en UTC en los archivos JSON, identifica cuándo se transformó la instantánea; no sustituye la fecha de observación ni representa una actualización automática del proveedor.

9. Diccionario físico: hechos horarios

fact_hourly | 43.800 filas

Campo	Tipo / clave	Descripción
source_row	INTEGER / PK	Identificador de fila asignado por el proceso de EA2
city_id	INTEGER / FK	Identificador dimensional local de EA1
time_id	INTEGER / FK	Identificador de la hora en la dimensión temporal de EA1
temperature_c	REAL	Temperatura meteorológica horaria de EA1
humidity_pct	REAL	Humedad relativa horaria
precipitation_mm	REAL	Precipitación del intervalo horario
wind_speed_kmh	REAL	Velocidad del viento horaria
energy_kwh	REAL	Valor energético sintético generado en EA1
synthetic_energy	INTEGER	Identifica la energía sintética
imputed_fields	TEXT	Campos imputados por EA2, separados por
quality_status	TEXT	Estado de calidad de EA2
is_outlier	INTEGER	Alguna variable quedó marcada como atípica por EA2
outlier_fields	TEXT	Variables señaladas, separadas por
outlier_temperature_c	INTEGER	Marca IQR de temperatura
outlier_humidity_pct	INTEGER	Marca IQR de humedad
outlier_precipitation_mm	INTEGER	Marca IQR de precipitación
outlier_wind_speed_kmh	INTEGER	Marca IQR de viento
outlier_energy_kwh	INTEGER	Marca IQR de energía sintética
temperature_zscore	REAL	Escalado de temperatura heredado de EA2, calculado por ciudad con todo 2025
energy_zscore	REAL	Escalado de energía heredado de EA2, calculado por ciudad con todo 2025
humidity_fraction	REAL	humidity_pct / 100
precipitation_log1p	REAL	ln(1 + precipitation_mm) sobre el valor numérico en mm
degrees_above_reference_c	REAL	max(temperature_c - reference_temperature_c, 0)
degrees_below_reference_c	REAL	max(reference_temperature_c - temperature_c, 0)
enrichment_status	TEXT	COMPLETE cuando los seis cruces tienen correspondencia y sus campos están presentes

source_row es PK y (city_id, time_id) es UNIQUE. Los índices por time_id y fecha solar favorecen consultas temporales. La vista enriched_data une las seis tablas y devuelve las 63 columnas en el mismo orden del CSV, incluyendo las marcas de calidad y de correspondencia. El SQL completo está en src/sql/schema.sql.

10. Herramientas y simulación de nube

Herramienta	Uso efectivo y justificación	Límite / siguiente paso
DuckDB 1.5.5	Conserva EA1 y la entrada limpia de enriquecimiento; consulta analítica SQL embebida.	No se presenta como un servicio cloud ni un clúster distribuido.
SQLite (sqlite3)	Modelo final portable con PK, FK y restricciones. Se abre sin administrar un servidor.	Un archivo local no resuelve alta concurrencia de escritura ni disponibilidad multizona.
Pandas 2.3.3 / NumPy 1.26.4	Lectura multiformato, reglas de calidad, cruces y reproducción de la simulación.	Carga en memoria. El volumen actual cabe en una máquina.
PySpark	Alternativa estudiada para crecimiento; no se usa ni se instala en esta entrega.	Su adopción requiere particiones, ejecutores y mediciones que justifiquen el costo operativo.
GitHub Actions	Runner Ubuntu/Python 3.12: instala, prueba, ejecuta y publica artefactos.	Es automatización remota; no reemplaza un almacén de datos persistente.
Openpyxl / lxml	Lectura y exportación XLSX; lectura de tablas HTML. XML usa el parser etree.	Los CSV y manifiestos conservan trazabilidad independiente del formato visual.

SQLite se incorpora para cumplir el modelo solicitado, sin describir retrospectivamente DuckDB como si fuera SQLite. La integración conserva ambas tecnologías con responsabilidades distintas: compatibilidad con las fases previas y consulta relacional final. La justificación es verificabilidad y portabilidad, no una supuesta superioridad universal (DuckDB Foundation, s. f.; SQLite, s. f.-c).

La simulación representa capas de almacenamiento con carpetas y archivos: JSON originales, datos limpios, datos enriquecidos y modelo consultable. GitHub aloja código e instantáneas; un runner ejecuta el lote. No se despliegan S3, Azure Blob Storage, redes virtuales ni nodos distribuidos. No se afirma haber procesado un volumen de Big Data real.

Pandas opera en memoria; para crecer se debe medir uso de RAM y tiempos, reducir columnas y evaluar procesamiento por particiones. Spark distribuye trabajo entre un controlador y ejecutores, pero incorporarlo ahora añadiría complejidad sin una necesidad demostrada (pandas development team, s. f.-b; Apache Software Foundation, s. f.).

11. Automatización y reproducción

El workflow activo está en `.github/workflows/bigdata.yml` de la raíz. La copia interior conserva la estructura académica `sebastian_clavijo_correa`; CI comprueba que ambas coincidan. Se ejecuta con `push`, `pull_request` o `workflow_dispatch` y solo solicita permiso `contents: read`. Las acciones están fijadas por SHA y las dependencias principales por versión.

Orden	Operación	Evidencia o condición
1	Checkout e instalación	Python 3.12; requirements.txt; setup.py
2	Pruebas unitarias	44 casos; build/tests.log
3	Replay y contraste EA1	5 JSON íntegros; valores idénticos
4	Limpieza y contraste EA2	CSV nuevo idéntico byte a byte
5	Enriquecimiento	6 cruces; CSV idéntico al antecedente
6	Modelo SQLite	6 tablas; 63 columnas reconstruidas
7	Publicación del artefacto	Salidas nuevas, auditorías, SQL, PDF y diagramas

La ejecución integrada escribe en `build/`, un directorio ignorado por Git y creado en el runner. Así no puede aprobar una evidencia versionada sin volver a generarla. Las instantáneas de entrada permanecen inalteradas. Si falla una comparación o restricción, el proceso devuelve error y no publica el artefacto exitoso.

11.1 Comandos desde la carpeta académica

```
python -m venv .venv
```

```
python -m pip install -r requirements.txt
```

```
python -m pip install -e . --no-deps --no-build-isolation
```

```
python -m unittest discover -s tests -v
```

```
python src/pipeline.py --output-dir build
```

```
python src/enrichment.py
```

Activa el entorno antes de instalar: en PowerShell, `.venv\Scripts\Activate.ps1`; en Linux/macOS, `source .venv/bin/activate`. El último comando permite ejecutar solo enriquecimiento desde `cleaned.duckdb`; la cadena completa incluye además ingesta, limpieza y SQLite. El README proporciona el comando de clonación y rutas de cada salida.

Los artefactos de Actions se retienen 30 días; no son una política de respaldo permanente. Las evidencias versionadas y el ZIP de entrega preservan el estado evaluado. `docs/ENTREGA.md` registra la URL de la ejecución remota y su resultado comprobado (GitHub, s. f.).

12. Resultados, conclusiones y mejoras

Comprobación local	Resultado observado
Regresión EA1	43.800 filas reproducidas con valores exactos
Regresión EA2 y enriquecimiento	Ambos CSV completos coinciden byte a byte
Preservación	32 variables heredadas intactas; 31 incorporadas
SQLite	5 ciudades, 365 fechas, 24 horas, 8.760 instantes, 1.825 hechos solares y 43.800 horarios
Integridad	0 huérfanos; integrity_check=ok; 2.759.400 celdas reconstruidas
Pruebas	44 casos aprobados: 13 de limpieza, 25 de enriquecimiento y 6 del modelo

Las pruebas del modelo introducen un valor solar diario inconsistente, un booleano inválido, una hora duplicada y una FK sin padre; verifican su rechazo. También comprueban los ceros iniciales en DIVIPOLA y la reconstrucción completa. Las pruebas previas ejercitan nulos, duplicados, unidades, claves de cruce, zonas horarias y conservación de datos. No se realizó una prueba de carga distribuida ni se midió una mejora predictiva.

12.1 Conclusiones

La arquitectura mantiene continuidad entre las entregas: se puede recorrer desde una fila enriquecida hasta la observación limpia y la extracción de EA1. Las comprobaciones de igualdad convierten esa trazabilidad en una condición ejecutable, no solo en una explicación. El modelo SQLite añade restricciones físicas que faltaban en el esquema original y separa las medidas diarias de las horarias.

La principal limitación es de alcance: cinco ciudades, un año y energía simulada no permiten inferir demanda real de Colombia. La repetición de datos de referencia, la ejecución en memoria y las instantáneas versionadas son manejables aquí, pero no constituyen una arquitectura productiva de gran escala.

12.2 Recomendaciones

Para una nube real, mover las instantáneas a almacenamiento de objetos versionado, usar Parquet particionado por fecha y revisar un almacén analítico administrado. Agregar autenticación con privilegio mínimo, gestión de secretos, respaldo y recuperación. Medir tiempos, RAM y concurrencia antes de migrar a PySpark. Incorporar ingesta incremental con reintentos, controles de esquema y un catálogo de procedencia.

Antes del modelado, obtener mediciones energéticas legítimas o mantener explícito el experimento sintético; dividir por tiempo, recalcular escalas en entrenamiento y excluir atributos diarios aún no disponibles al momento de predecir. Las mejoras propuestas son trabajo futuro, no capacidades ya desplegadas.

Bibliografía

Apache Software Foundation. (s. f.). *Cluster mode overview*.
<https://spark.apache.org/docs/latest/cluster-overview.html>

DuckDB Foundation. (s. f.). *Why DuckDB*.
https://duckdb.org/why_duckdb

GitHub. (s. f.). *Workflow artifacts*.
<https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>

Open-Meteo. (s. f.). *Historical weather API*.
<https://open-meteo.com/en/docs/historical-weather-api>

pandas development team. (s. f.-a). *pandas.DataFrame.merge*.
<https://pandas.pydata.org/pandas-docs/version/2.3/reference/api/pandas.DataFrame.merge.html>

pandas development team. (s. f.-b). *Scaling to large datasets*.
https://pandas.pydata.org/pandas-docs/version/2.3/user_guide/scale.html

SQLite. (s. f.-a). *SQLite foreign key support*.
<https://www.sqlite.org/foreignkeys.html>

SQLite. (s. f.-b). *Datatypes in SQLite*.
<https://www.sqlite.org/datatype3.html>

SQLite. (s. f.-c). *Appropriate uses for SQLite*.
<https://www.sqlite.org/whentouse.html>

Clavijo Correa, J. S. (2026a). *Preprocesamiento y limpieza de datos en plataforma de Big Data en la nube* [Código fuente y evidencias]. GitHub.
https://github.com/JSebastianCCorrea/Preprocesamiento_Limpieza_de_Datos_en_Plataforma_de_Big_Data_en_la_Nube

Clavijo Correa, J. S. (2026b). *Enriquecimiento de datos en plataforma Big Data en la nube* [Código fuente y evidencias]. GitHub.
https://github.com/JSebastianCCorrea/Enriquecimiento_Datos_Plataforma_Big_Data_Nube

Materiales y fuentes de datos

La revisión funcional usa el código y las evidencias locales de EA1, EA2 y enriquecimiento. La consigna y su imagen de estructura fueron suministradas en el chat; los enlaces del aula requieren acceso y no se atribuye contenido adicional no consultado. FUENTES.md y src/sources/source_manifest.json identifican las consultas DANE, Nager.Date y Open-Meteo, sus licencias, formatos preparados y doce respuestas conservadas. No se reproducen credenciales ni datos personales.